FATEC VOTORANTIM
Curso Superior de Tecnologia em Ciência de Dados para Negócios

Ana Elisa Rubinato Cavalheiro
Arthur Nunes dos Santos
Bruno Felipe de Souza Araujo
Caio Cesar Corrá
Lucas Camelo
Nicole Fava Pires

DEPLOY GUIDE

Inteligência de Risco em Comodato de Chopp

Projeto Integrador VI

Votorantim
2026

Controle de Versão

Registro histórico de todas as revisões deste documento, com data, responsável e descrição da alteração.

Tabela 1 - Controle de Versão

Versão	Data	Autor	Descrição
1.0	16/04/2026	Equipe PI5	Criação inicial do documento
1.1	16/05/2026	Equipe PI5	Revisão do site e simulação do modelo preditivo
1.2	01/06/2026	Equipe PI5	Ajustes para deploy em PythonAnywhere
1.3	25/05/2026	Equipe PI5	Ajustes pós revisão
2.0	05/10/2026	Equipe PI6	Atualização para o PI6: dados e modelo no Databricks, fonte de dados Local ou Databricks, login e adequação à LGPD, nova regra de risco e novo modelo

Fonte: Autoria própria

DEPLOY GUIDE

Controle de Versão **1**
1. Objetivo **4**
2. Escopo **4**
3. O Que Mudou Desde a Versão 1.3 ..... **4**
4. Definições e Siglas **5**
5. Visão Geral da Solução **6**
5.1 Contexto de Negócio 6
5.2 Regra de Risco e População 6
5.3 Modelo em Produção 6
5.4 Fluxo da Solução 7
6. Arquitetura da Solução **7**
6.1 Componentes 7
6.2 Solução Adotada para Servir o Modelo 8
6.3 Artefatos de Produção 8
7. Estrutura do Site **8**
8. Pré-requisitos **9**
8.1 Databricks 9
8.2 Site 9
9. Preparação dos Dados e do Modelo **10**
10. Deploy do Site no PythonAnywhere **10**
10.1 Envio dos Arquivos 10
10.2 Ambiente Virtual e Dependências 11
10.3 Web App e Arquivo WSGI 11
10.4 Credenciais do Databricks 11
10.5 Primeiro Usuário e Inicialização 12
11. Fonte de Dados: Local ou Databricks **12**
12. Variáveis de Ambiente **13**
13. Usuários, Segurança e LGPD **14**
13.1 Perfis e Gestão de Usuários 14
13.2 Controles Implementados 14
14. Versionamento **14**
14.1 Versão dos Dados e Nomes 14
14.2 Conteúdo do Model Card 15
14.3 Experimentos no MLflow 15
15. Atualização e Troca do Modelo **15**
15.1 Quando Retreinar 15
15.2 Ciclo de Atualização 16
16. Limiar e Faixas de Risco **16**
17. Testes **17**
17.1 Saúde da Aplicação 17
17.2 Acesso e Segurança 17
17.3 Simulador 17
17.4 Fonte de Dados 18
18. Monitoramento **18**
19. Riscos e Mitigações **18**

20. Limitações Conhecidas 19
21. Próximos Passos Recomendados 19
22. Responsáveis 20
23. Aprovação 20

1. Objetivo

Este guia descreve como implantar, atualizar e manter a solução de Inteligência de Risco em Comodato de Chopp da Chopp & Cia, desenvolvida no Projeto Integrador VI da FATEC Votorantim. A solução estima, para cada cliente, a probabilidade de atrasar o pagamento das parcelas e a devolução da chopeira, e apresenta o resultado em um site com acesso restrito.

Em relação à versão 1.3 (PI5), o preparo dos dados, o treinamento e o registro do modelo passaram para o Databricks. O site continua em Flask, mas agora lê o modelo e a carteira de clientes diretamente do Databricks (ou de arquivos locais, como contingência), exige login para qualquer dado da empresa e registra as consultas.

2. Escopo

Este documento cobre:

- Preparação do ambiente Databricks: catálogo, volumes, tabelas e SQL warehouse
- Execução dos notebooks que geram a base de dados e o modelo de produção
- Publicação do site no PythonAnywhere e escolha da fonte de dados (Local ou Databricks)
- Gestão de usuários, segurança do acesso e trilha de auditoria
- Versionamento, troca do modelo em produção e ciclo de retreinamento
- Testes após o deploy, monitoramento, riscos e limitações conhecidas

3. O Que Mudou Desde a Versão 1.3

Resumo das mudanças entre o que foi entregue no PI5 e a solução atual.

Tabela 1 - Evolução PI5 → PI6

Tema	Versão 1.3 (PI5)	Versão 2.0 (PI6)
Fonte dos dados	Backup do SQL Server (jan/2026) lido por 3 views do ERP	Dump do ERP (ago/2026) lido em Python com uma lista fechada de colunas
Filtros do ERP	Nenhum	Registro ativo, pedido cancelado, só operações de venda e pessoa que é cliente. Antes, retirada de chopeira era contada como compra
Plataforma	Um único notebook local	8 notebooks; dados em tabelas Delta no Unity Catalog, experimentos no MLflow, modelo no Model Registry
Regra de risco	Atraso > 20% no pagamento OU no comodato	Atraso > 20% no pagamento E no comodato
Clientes no treino	Só quem tinha mais de 2 compras	Todo cliente com ao menos 1 compra
Variáveis	12, incluindo a média de dias de atraso	13, sem as médias de dias de atraso e com 3 novas do ERP
Escolha do modelo	AUC e Recall	MCC, medido em uma grade de 216 combinações de decisões
Desempenho	AUC 0,992 (inflado: o modelo via parte da resposta)	AUC 0,842 e MCC 0,466 em clientes não vistos

Tema	Versão 1.3 (PI5)	Versão 2.0 (PI6)
Saídas	Arquivo .pkl	.pkl com ficha do modelo, carteira escorada em tabela Delta e modelo registrado
Site	Aberto; nome do cliente na tela	Login para dados da empresa, sem nome de cliente, trilha de auditoria e fonte de dados Local ou Databricks

Fonte: Autoria própria

> **Por que o AUC caiu de 0,992 para 0,842**
> O modelo do PI5 recebia a média de dias de atraso, que é maior que zero sempre que a taxa de atraso que define o risco também é. Parte do acerto vinha de reproduzir a própria regra. Sem essas colunas, o número é menor, mas mede o que o modelo realmente consegue estimar a partir do comportamento do cliente.

4. Definições e Siglas

Tabela 2 - Definições e Siglas

Termo	Descrição
Databricks	Plataforma de dados em nuvem onde ficam as tabelas, os notebooks, os experimentos e o modelo
Unity Catalog	Catálogo do Databricks que organiza tabelas e volumes em catálogo.schema.objeto
Tabela Delta	Formato de tabela do Databricks, com histórico de versões
Volume	Pasta de arquivos do Unity Catalog (ex.: /Volumes/projetointegrador/projetointegrador/artefatos)
SQL warehouse	Recurso do Databricks que executa consultas SQL; o site o usa para ler a carteira
MLflow	Ferramenta, criada pela Databricks, que registra experimentos, métricas e modelos
Model Registry	Registro de modelos do MLflow, com versões numeradas
Pacote do modelo (.pkl)	Arquivo com o pipeline treinado e a ficha do modelo (model card)
Model card	Ficha do modelo: variáveis, regra de risco, métricas, versão dos dados e data de geração
Carteira escorada	Tabela com a probabilidade e a faixa de risco de cada cliente
DATA_VERSION	Versão dos dados; nomeia arquivos e tabelas (ex.: v1_0) para que nada seja sobrescrito
ID_PESSOA	Identificador do cliente no ERP; única identificação usada pela solução
MCC	Coeficiente de correlação de Matthews, de −1 a 1. Só sobe quando as duas classes são bem classificadas; é a métrica de escolha do modelo
AUC-ROC	Capacidade de ordenar clientes por risco, de 0,5 (acaso) a 1,0
Limiar (threshold)	Probabilidade a partir da qual o cliente é classificado como alto risco (0,50)

Termo	Descrição
PythonAnywhere	Serviço de hospedagem de aplicações Python onde o site é publicado
WSGI	Interface entre o servidor web e a aplicação Flask

Fonte: Autoria própria

5. Visão Geral da Solução

5.1 Contexto de Negócio

A Chopp & Cia é uma distribuidora de chopp em Ponta Porã (MS), na fronteira com o Paraguai. Boa parte dos clientes compra a prazo e recebe a chopeira em comodato, então cada venda carrega dois riscos:

- **Risco financeiro:** atraso no pagamento das parcelas
- **Risco de ativo (comodato):** devolução da chopeira fora do prazo

5.2 Regra de Risco e População

O modelo é uma classificação binária supervisionada. O alvo segue a regra abaixo:

```
ALTO_RISCO = 1  se  TAXA_ATRASO_PAGAMENTO > 20%  E  TAXA_ATRASO_COMODATO > 20%
ALTO_RISCO = 0  caso contrário

População de treino: todo cliente com ao menos 1 compra (1.281 clientes, 53,7% na classe
1)
```

Com “OU”, como no PI5, cerca de 90% da carteira seria classificada como risco e a classe de bons pagadores ficaria pequena demais para o modelo aprender. O corte antigo de mais de 2 compras também deixou de ser viável: com a contagem corrigida, sobrariam 332 clientes e apenas 6 casos na classe rara. As duas decisões foram medidas na grade de cenários do notebook 08.

5.3 Modelo em Produção

Tabela 3 - Modelo em produção

Item	Valor
Algoritmo	Random Forest (200 árvores, profundidade máxima 8, mínimo de 2 clientes por folha, classes balanceadas)
Pré-processamento	StandardScaler nas numéricas; OneHotEncoder nas categóricas (categoria desconhecida é ignorada)
Variáveis numéricas (10)	FREQUENCIA_COMPRAS, TICKET_MEDIO, TOTAL_GASTO, DIAS_DESDE_PRIMEIRA_COMPRA, DIAS_DESDE_ULTIMA_COMPRA, TOTAL_PARCELAS, TOTAL_COMODATOS, PCT_COMPRAS_A_PRAZO*, PRAZO_MEDIO_COMODATO*, VALOR_MEDIO_COMODATO*
Variáveis categóricas (3)	PERFIL, CIDADE, PAGAMENTO
Fora do modelo, de propósito	MEDIA_DIAS_ATRASO_PAG/COM (mesma origem do alvo), componentes do alvo, bloqueio do ERP, nome, CPF, CNPJ e endereço

Item	Valor
Desempenho em clientes não vistos (30%)	MCC 0,466 · AUC 0,842 · Recall 0,715 · Precisão 0,771 · F1 0,742
Desempenho na validação cruzada (5 folds)	MCC 0,485 · AUC 0,840
Arquivo final	Retreinado com 100% da população após a avaliação

Fonte: Autoria própria

* Variáveis novas no PI6, incluídas depois de medido o preenchimento real no ERP.

5.4 Fluxo da Solução

1. **Notebook 01 (local):** lê o dump do ERP, aplica os filtros e gera o consolidado por cliente (CSV)
2. **Volume "dados":** o CSV é enviado ao Databricks
3. **Notebook 02:** publica o consolidado como tabela Delta versionada no Unity Catalog
4. **Notebook 03:** análise exploratória; grava os gráficos do painel no volume "artefatos"
5. **Notebook 08:** compara os cenários, treina o modelo de produção e grava o pacote (.pkl), a carteira escorada (Delta e parquet) e o registro no Model Registry
6. **Site:** carrega o pacote e a tabela de clientes, calcula o score de cada cliente e exibe painel e simulador
7. **Analista:** consulta o score e decide; a decisão final é sempre de uma pessoa

6. Arquitetura da Solução

6.1 Componentes

Tabela 4 - Componentes

Componente	Tecnologia	Função
Extração	Notebook 01 · Python local	Lê o dump .sql do ERP e gera o consolidado por ID_PESSOA
Dados	Databricks · Delta · Unity Catalog	Tabela versionada do consolidado e da carteira escorada
Modelagem	Notebooks 03–08 · scikit-learn	Análise exploratória, grade de cenários e modelo de produção
Experimentos	MLflow	24 experimentos da grade, estudos de hiperparâmetro e o run de produção
Modelo	Volume + Model Registry	Pacote .pkl para o site e versão registrada no MLflow
Site	Flask 3.1 · PythonAnywhere	Apresentação pública, painel e simulador com login
Acesso	Flask-Login · SQLite	Usuários, senhas com hash, bloqueio e trilha de auditoria

Fonte: Autoria própria

6.2 Solução Adotada para Servir o Modelo

O modelo é treinado e registrado no Databricks. O site lê do Databricks o pacote do modelo (Volume, via Files API) e a tabela de clientes (via SQL warehouse) e executa o pipeline no próprio servidor do site, que já recebe a carteira inteira escorada. Os arquivos locais são mantidos como contingência: se o Databricks estiver indisponível na inicialização, o site sobe com a fonte local e avisa o administrador.

O consumo do modelo por um endpoint do Model Serving do Databricks não foi adotado nesta versão e permanece como evolução futura (Seção 21).

6.3 Artefatos de Produção

Cada artefato tem um único notebook responsável:

Tabela 5 - Artefatos de produção

Artefato	Gerado por	Databricks	Local (contingência)
Consolidado por cliente	01 → 02	tabela dataset_consolidado_v1_0	Desenvolvimento/data/ dataset_consolidado_v1_0.csv
Pacote do modelo	08	artefatos/modelo_campeao_v1_0 .pkl	Site/modelo/ modelo_campeao_v1_0.pkl
Carteira escorada	08	tabela carteira_escorada_v1_0 e .parquet	Desenvolvimento/data/
Modelo registrado	08	workspace.default.chopp_risco_c ampeao	—
Gráficos do painel	03	artefatos/graficos/	Site/restrito/graficos/
Base do simulador	preparar_dad os.py	montada a partir da tabela	Site/data/ carteira_site_v1_0.csv

Fonte: Autoria própria

Todas as tabelas ficam em projetointegrador.projetointegrador. A view dataset_consolidado_corrente aponta para a carga mais recente; experimentos e o site usam sempre a tabela versionada.

7. Estrutura do Site

Organização da pasta Site/ no servidor. Itens marcados com * não estão no repositório e precisam ser enviados ou gerados no servidor.

```
Site/
├── app.py                  # rotas, simulador, painel, /health
├── fontes.py               # fonte Local e fonte Databricks
├── auth.py                 # login, bloqueio e trilha de auditoria
├── compat.py               # ajuste de ambiente carregado antes do scikit-learn
├── preparar_dados.py       # gera a base do simulador (fonte local)
├── gerenciar_usuarios.py   # criação e manutenção de usuários
├── requirements.txt
├── databricks.exemplo.json # modelo de configuração, sem segredos
├── templates/  static/     # páginas, CSS e JavaScript
├── modelo/                 # * modelo_campeao_v1_0.pkl
├── data/                   # * carteira_site_v1_0.csv
├── restrito/graficos/      # * gráficos do painel (servidos só com login)
└── instance/               # * usuarios.db, secret_key, fonte.json, databricks.json
```

Atenção

Nada em static/ é protegido: qualquer arquivo ali é público por URL. Gráficos com dados da empresa ficam em restrito/graficos/.

A pasta instance/ guarda senhas (com hash), a chave da sessão e o token do Databricks. Não versionar e não compartilhar.

8. Pré-requisitos

8.1 Databricks

- Workspace com Unity Catalog (o projeto foi desenvolvido no Databricks Free Edition, computação serverless)
- Catálogo e schema projetointegrador.projetointegrador
- Volumes dados (entrada do CSV) e artefatos (pacote, carteira escorada e gráficos), criados antes de rodar os notebooks
- Um SQL warehouse, ligado quando o site for carregar ou trocar de fonte
- Um token de acesso pessoal com permissão de leitura na tabela e no volume

8.2 Site

Python 3.11, 3.12 ou 3.13 (o numpy 2.3 exige 3.11 ou superior). O requirements.txt fixa as versões e instala apenas pacotes pré-compilados:

Tabela 6 - Bibliotecas do site

Biblioteca	Versão	Finalidade
Flask	3.1.2	Aplicação web
Werkzeug	3.1.3	Utilitários do Flask e hash de senhas
Flask-Login	0.6.3	Sessão e autenticação
requests	2.33.1	Chamadas às APIs do Databricks
scikit-learn	1.7.2	Execução do pipeline treinado
numpy	2.3.2	Operações numéricas
pandas	2.3.1	Leitura e montagem da carteira

Fonte: Autoria própria

Versão do scikit-learn

O .pkl só carrega com a mesma versão do scikit-learn que o gerou. Os pacotes recentes registram essa versão no model card, e a tela de fonte de dados avisa quando ela diverge da do site. Se o notebook 08 rodar com outra versão, alinhe o requirements.txt antes de ativar o modelo.

9. Preparação dos Dados e do Modelo

Ordem de execução para publicar uma versão de dados. Cada notebook começa por uma célula de parâmetros; confira DATA_VERSION em todos antes de executar.

1	**Extração do ERP (notebook 01, Windows local)** Informe o caminho do dump .sql. O notebook aplica os filtros do ERP, lê só as colunas previstas e interrompe a execução se aparecer alguma coluna de nome ou documento. Saída: dataset_consolidado_v1_0.csv.
2	**Envio ao Databricks** Copie o CSV para /Volumes/projetointegrador/projetointegrador/dados/.
3	**Ingestão (notebook 02, Databricks)** Publica a tabela dataset_consolidado_v1_0 com dicionário de colunas e atualiza a view dataset_consolidado_corrente. Recusa regravar uma versão já publicada.
4	**Análise exploratória (notebook 03)** Com SALVAR_FIGURAS ligado, grava os 11 gráficos do painel em artefatos/graficos/ com os nomes que o site procura. Não renomeie esses arquivos.
5	**Cenários e modelo de produção (notebook 08)** Executa a grade de cenários no MLflow, treina o modelo escolhido, mede em 30% dos clientes, retreina com 100% e grava modelo_campeao_v1_0.pkl e carteira_escorada_v1_0.parquet no volume, a tabela carteira_escorada_v1_0 e a versão no Model Registry.

Os notebooks 04 a 07 servem para estudo de hiperparâmetros e não são necessários para atualizar a produção: só o notebook 08 gera o pacote do modelo.

Proteções contra sobrescrita: os notebooks 01, 02 e 08 recusam regravar arquivos ou tabelas de uma versão já existente. Para refazer a mesma versão, use SOBRESCREVER = True de forma consciente; o caminho normal é incrementar DATA_VERSION.

10. Deploy do Site no PythonAnywhere

10.1 Envio dos Arquivos

Clone o repositório em um console Bash ou envie a pasta Site/ pela aba Files. Os arquivos marcados com * na Seção 7 não estão no repositório:

- modelo/modelo_campeao_v1_0.pkl — copiado do volume artefatos ou de Desenvolvimento/data/
- data/carteira_site_v1_0.csv — gerado localmente com python preparar_dados.py (só as 13 variáveis e o ID_PESSOA; nenhum nome)
- restrito/graficos/ — os 11 gráficos do notebook 03

Esses três itens só são usados pela fonte local. Com a fonte Databricks, o site os lê do workspace.

10.2 Ambiente Virtual e Dependências

```
mkvirtualenv --python=python3.11 chopp
cd ~/CDN_ProjetoIntegrador6/Site
pip install --no-cache-dir --no-compile -r requirements.txt
```

No PythonAnywhere, a imagem de sistema “haggis” oferece Python até 3.13 e a “innit” até 3.12.

10.3 Web App e Arquivo WSGI

1. Na aba Web, clique em Add a new web app e escolha Manual configuration com a mesma versão do Python
2. Em Source code, informe /home/<usuario>/CDN_ProjetoIntegrador6/Site
3. Em Virtualenv, informe o ambiente criado (ex.: /home/<usuario>/.virtualenvs/chopp)
4. Ative Force HTTPS
5. Abra o arquivo WSGI e substitua todo o conteúdo pelo bloco abaixo

```
import os, sys
PROJETO = '/home/<usuario>/CDN_ProjetoIntegrador6/Site'
if PROJETO not in sys.path:
    sys.path.insert(0, PROJETO)
os.chdir(PROJETO)

os.environ['SECRET_KEY'] = '<texto aleatório longo>'
os.environ['COOKIE_SEGURO'] = '1'
os.environ['FONTE_DADOS'] = 'databricks'

from app import application
```

As variáveis de ambiente são definidas no próprio arquivo WSGI, antes da importação do app. A lista completa está na Seção 12.

10.4 Credenciais do Databricks

Copie databricks.exemplo.json para instance/databricks.json e preencha:

```
{
  "host": "https://<workspace>.cloud.databricks.com",
  "token": "<token de acesso pessoal>",
  "warehouse_id": "<id do SQL warehouse>",
  "tabela": "projetointegrador.projetointegrador.dataset_consolidado_v1_0",
  "volume": "/Volumes/projetointegrador/projetointegrador/artefatos",
  "arquivo_modelo": "modelo_campeao_v1_0.pkl",
  "pasta_graficos": "graficos"
}
```

Alternativamente, use as variáveis DATABRICKS_HOST, DATABRICKS_TOKEN e DATABRICKS_WAREHOUSE_ID (as variáveis de ambiente têm precedência sobre o arquivo). O token nunca é digitado no site nem gravado em log.

> **Acesso externo no PythonAnywhere**
> Contas gratuitas do PythonAnywhere só acessam uma lista restrita de domínios externos. Confirme que o endereço do workspace Databricks é alcançável pelo plano contratado; caso contrário, o site sobe com a fonte local.

10.5 Primeiro Usuário e Inicialização

```
cd ~/CDN_ProjetoIntegrador6/Site
python gerenciar_usuarios.py criar admin --admin
```

Clique em Reload na aba Web e acesse https://<usuario>.pythonanywhere.com/health. Em seguida, entre com o usuário admin e abra menu do usuário → Fonte de dados para conferir a fonte ativa (Seção 11). A pasta instance/ precisa de permissão de escrita.

11. Fonte de Dados: Local ou Databricks

O administrador escolhe a fonte em menu do usuário → Fonte de dados (/admin/fonte). As duas fontes entregam ao site o mesmo conteúdo e montam a carteira com a mesma regra.

Tabela 7 - Fontes de dados

	Local	Databricks
Modelo	modelo/modelo_campeao_v1_0.pkl	<volume>/modelo_campeao_v1_0.pkl (Files API)
Carteira	data/carteira_site_v1_0.csv	tabela Delta do notebook 02 (SQL warehouse)
Gráficos	restrito/graficos/	<volume>/graficos/ (Files API, em cache)

Fonte: Autoria própria

- **Testar** carrega a fonte sem ativá-la e mostra clientes, algoritmo, versão dos dados, tempo de carga e avisos
- **Ativar** só troca a fonte se a carga der certo; se falhar, a fonte atual continua no ar
- A escolha fica em instance/fonte.json e vale nas próximas inicializações
- Se o Databricks estiver fora do ar ao subir o site, ele usa a fonte local e exibe o motivo ao administrador

12. Variáveis de Ambiente

Tabela 8 - Variáveis de ambiente

Variável	Padrão	Uso
SECRET_KEY	gerada em instance/secret_key	Assinatura da sessão
COOKIE_SEGURO	0	Use 1 em produção (HTTPS)
LOGIN_SITE_INTEIRO	0	1 exige login também na apresentação pública
SESSAO_MINUTOS	15	Expiração da sessão
LOGIN_MAX_TENTATIVAS	5	Tentativas erradas antes do bloqueio
LOGIN_MINUTOS_BLOQUEIO	15	Duração do bloqueio
LOG_RETENCAO_DIAS	180	Eventos mais antigos são apagados na inicialização
FONTE_DADOS	local	Fonte inicial, se instance/fonte.json não existir
MODEL_PATH, DATASET_PATH	pastas do site	Arquivos da fonte local
AUTH_DB_PATH	instance/usuarios.db	Banco de usuários e auditoria
DATABRICKS_HOST, _TOKEN, _WAREHOUSE_ID	—	Conexão com o Databricks
DATABRICKS_TABELA, _VOLUME, _ARQUIVO_MODELO, _PASTA_GRAFICOS	—	Opcionais; substituem o databricks.json

Fonte: Autoria própria

O limiar de classificação não é mais uma variável de ambiente, como no PI5: ele vem do model card (Seção 16).

13. Usuários, Segurança e LGPD

13.1 Perfis e Gestão de Usuários

Não há cadastro pelo site. O administrador cria e mantém os usuários no console do servidor:

```
python gerenciar_usuarios.py listar
python gerenciar_usuarios.py criar maria            # perfil analista
python gerenciar_usuarios.py criar joao --admin     # perfil administrador
python gerenciar_usuarios.py senha maria
python gerenciar_usuarios.py desbloquear maria
python gerenciar_usuarios.py desativar maria        # ativar maria reverte
```

Tabela 9 - Perfis de acesso

Perfil	Acesso
Visitante (sem login)	Apresentação: problema, evolução PI5 → PI6, arquitetura, variáveis, cenários, métricas do modelo e LGPD
Analista	Tudo acima, mais o painel da carteira e o simulador
Administrador	Tudo acima, mais a trilha de auditoria e a escolha da fonte de dados

Fonte: Autoria própria

13.2 Controles Implementados

- Números da empresa (tamanho da carteira, faturamento, percentuais, gráficos) não são enviados à página sem login
- Nenhum nome de cliente sai do ERP: a identificação é só o ID_PESSOA (dado pseudonimizado, art. 13 § 4º)
- Senhas com hash, sessão de 15 minutos e bloqueio após 5 tentativas erradas
- Proteção CSRF nos formulários e cabeçalhos de segurança; respostas da API não ficam em cache
- Trilha de auditoria com login, logout, bloqueios, consultas por ID, simulações e trocas de fonte (art. 37), apagada após 180 dias
- O score apoia a decisão; a liberação final é de uma pessoa (art. 20)

14. Versionamento

14.1 Versão dos Dados e Nomes

DATA_VERSION identifica a carga de dados e entra no nome de todos os artefatos. Incremente-a ao trocar o dump do ERP ou qualquer regra de negócio. RANDOM_STATE = 42 é a semente única do projeto: rodar de novo a mesma versão produz os mesmos números.

```
dataset_consolidado_v1_0      tabela Delta e CSV
modelo_campeao_v1_0.pkl       pacote do modelo
carteira_escorada_v1_0        tabela Delta e .parquet
carteira_site_v1_0.csv        base do simulador (fonte local)
workspace.default.chopp_risco_campeao   versões 1, 2, 3... no Model Registry
```

14.2 Conteúdo do Model Card

Tabela 10 - Campos do model card

Campo	Conteúdo
cenario	População, regra do alvo e conjunto de variáveis escolhidos na grade
algoritmo / hiperparametros	Classe do estimador e parâmetros
features_numericas / features_categoricas	Lista de variáveis; o site monta o formulário a partir dela
threshold_classificacao	Limiar da classificação 0/1 (0,50)
metricas_cv / metricas_holdout	MCC, AUC, recall, precisão, F1 e acurácia balanceada
matriz_holdout / n_holdout	Matriz de confusão e tamanho do conjunto de teste
n_treino_final / prevalencia	Clientes no treino final e proporção da classe de risco
sklearn_version	Versão do scikit-learn que gerou o pacote
data_version / random_state / gerado_em	Rastreabilidade da execução

Fonte: Autoria própria

14.3 Experimentos no MLflow

O notebook 08 cria uma pasta no MLflow com um experimento para cada combinação de população × regra de risco (24 no total). Dentro de cada experimento, cada run é uma combinação de conjunto de variáveis × algoritmo, com parâmetros, métricas e tags. O modelo de produção fica no experimento _PRODUCAO, com o model card anexado. Os notebooks 05 a 07 registram seus estudos de hiperparâmetro, mas não promovem modelo.

15. Atualização e Troca do Modelo

15.1 Quando Retreinar

- A cada nova extração do ERP (recomendado: trimestral)
- Quando a distribuição das faixas de risco mudar de forma persistente (Seção 18)
- Quando houver mudança de negócio: nova região, nova política de crédito ou de comodato

15.2 Ciclo de Atualização

1. Gere um novo dump do ERP e incremente DATA_VERSION (ex.: 1.1) nos notebooks
2. Execute 01 → envio do CSV → 02 → 03 → 08 (Seção 9)
3. Compare o MCC em clientes não vistos com o do modelo atual (0,466). Diferenças menores que ±0,047 (o desvio entre folds) não indicam melhora nem piora
4. Atualize tabela e arquivo_modelo no databricks.json (ou as variáveis de ambiente) para a nova versão
5. Em Fonte de dados, clique em Testar; confira clientes, versão e avisos; depois clique em Ativar
6. Para a fonte local, copie o novo .pkl para modelo/, rode preparar_dados.py com o novo consolidado e ajuste MODEL_PATH e DATASET_PATH
7. Atualize os números fixos do painel em PAINEL_EMPRESA (app.py) com as saídas do notebook 03
8. Execute os testes da Seção 17 e registre a troca no Controle de Versão

Os artefatos da versão anterior continuam no volume e no Model Registry. Para voltar atrás, aponte a configuração para a versão anterior e ative-a novamente.

16. Limiar e Faixas de Risco

O site exibe três faixas de leitura do score e uma classificação binária:

Tabela 11 - Faixas de risco

Faixa	Probabilidade	Orientação exibida no site
BAIXO	abaixo de 35%	Liberar dentro da política padrão
MÉDIO	de 35% a 65%	Exigir entrada ou acompanhar de perto
ALTO	65% ou mais	Exigir garantias antes de liberar

Fonte: Autoria própria

- A classificação 0/1 usa o threshold_classificacao do model card (0,50)
- As faixas 35% e 65% estão em app.py (FAIXA_BAIXO, FAIXA_ALTO) e no notebook 08, que grava FAIXA_RISCO na carteira escorada. Para mudá-las, altere os dois lugares juntos, ou o site e a tabela passam a divergir
- Mudar faixas ou limiar não exige retreinar o modelo

17. Testes

17.1 Saúde da Aplicação

```
GET https://<usuario>.pythonanywhere.com/health

{ "status": "ok", "fonte": "databricks", "modelo_carregado": true,
  "carteira_carregada": true, "algoritmo": "RandomForestClassifier",
  "data_version": "1.0" }
```

"status": "degradado" indica que o modelo ou a carteira não carregaram; o motivo aparece na tela Fonte de dados para o administrador.

17.2 Acesso e Segurança

Tabela 12 - Testes de acesso

Teste	Resultado esperado
Abrir o site sem login	Apresentação visível; painel e simulador pedem login; nenhum número da carteira na página
Abrir /graficos/composicao_carteira.png sem login	Redireciona para a tela de login
Chamar /api/predict sem login	HTTP 401 com mensagem de sessão expirada
Errar a senha 5 vezes	Conta bloqueada por 15 minutos; evento na trilha de auditoria
Usuário analista abrir /admin/fonte	Acesso negado

Fonte: Autoria própria

17.3 Simulador

Com login, teste os perfis abaixo no modo Cliente novo. Os valores foram calculados com o modelo v1.0 e devem se repetir enquanto o mesmo pacote estiver ativo.

Tabela 13 - Testes do simulador

Cenário	Dados informados	Resultado esperado
Baixo risco	1 compra de R$ 400 há 500 dias; 1 parcela; 1 comodato de R$ 9.400 por 3 dias; 0% a prazo; Física; Ponta Porã; PIX	BAIXO, cerca de 6%
Risco médio	1 compra de R$ 480 há 260 dias (1ª há 280); 1 parcela; 1 comodato de R$ 9.400 por 3 dias; 100% a prazo; Física; Ponta Porã; PIX	MÉDIO, cerca de 43%
Alto risco	6 compras, R$ 3.000 no total, última há 60 dias (1ª há 700); 8 parcelas; 5 comodatos de R$ 9.800 por 3 dias; 60% a prazo; Física; Ponta Porã; Dinheiro	ALTO, cerca de 99%
ID inexistente	ID_PESSOA que não está na carteira	Mensagem “ID não está na carteira”
Valor inválido	Compras a prazo = 150%	Campo marcado como inválido; nada é calculado

Fonte: Autoria própria

Na busca por ID, confira também se o cartão mostra a comparação com a regra de negócio e se a consulta aparece na trilha de auditoria.

17.4 Fonte de Dados

- Testar a fonte Databricks: deve informar a quantidade de clientes da carteira e DATA_VERSION 1.0, sem avisos de versão
- Desligar o SQL warehouse e clicar em Ativar: a troca deve falhar e a fonte atual continuar no ar

18. Monitoramento

Tabela 14 - Monitoramento

Indicador	Frequência	Ponto de atenção
Distribuição das faixas na carteira escorada	A cada carga	Na v1.0: 31% BAIXO, 39% MÉDIO, 30% ALTO. Desvio persistente sugere mudança no perfil dos clientes
MCC do novo modelo	A cada retreino	Queda maior que 0,047 em relação ao modelo atual
Peso de PERFIL e CIDADE no modelo	A cada retreino	Podem indicar nacionalidade de forma indireta (hoje 1,5% e 2,5%); aumento exige revisão
Trilha de auditoria	Mensal	Consultas fora do padrão, logins bloqueados, trocas de fonte
Site e /health	Contínuo	status diferente de "ok" ou erros no error.log do PythonAnywhere
Usuários ativos	Trimestral	Desativar quem não precisa mais de acesso

Fonte: Autoria própria

19. Riscos e Mitigações

Tabela 15 - Riscos e mitigações

Risco	Prob.	Impacto	Mitigação
Versão do scikit-learn diferente entre o notebook 08 e o site	Média	Alto	Versões fixadas no requirements.txt; o teste de fonte compara com a versão do model card
Databricks ou SQL warehouse indisponível	Média	Médio	Ativação só troca se a carga der certo; na inicialização, contingência com a fonte local
Vazamento do token do Databricks	Baixa	Alto	Token fora do repositório (instance/ ou variável de ambiente), nunca exibido no site; renovar periodicamente
Acesso indevido a dados de clientes	Baixa	Alto	Login, bloqueio, sessão curta, perfis e trilha de auditoria
Perfil dos clientes muda com o tempo	Alta	Alto	Monitorar as faixas a cada carga; retreinar trimestralmente
Números do painel desatualizados	Média	Baixo	Atualizar PAINEL_EMPRESA no ciclo de retreino (Seção 15.2)
Gráfico com dado da empresa publicado em static/	Baixa	Alto	Gráficos só em restrito/graficos/ ou no volume

Fonte: Autoria própria

20. Limitações Conhecidas

- **Sem corte temporal:** variáveis e alvo vêm da mesma janela de tempo. O modelo identifica o perfil de quem atrasa, mas não foi testado em prever atrasos futuros
- **Faixa de incerteza:** o MCC varia ±0,047 entre folds; diferenças menores não indicam superioridade
- **Classe de risco majoritária (53,7%):** acurácia e AUC isoladas enganam; por isso a escolha é por MCC
- **Clientes sem compra:** são escorados, mas não participaram do treino; o cartão do simulador indica isso
- **Cobertura regional:** o modelo foi construído com clientes de Ponta Porã e região

21. Próximos Passos Recomendados

Das recomendações do PI5, foram concluídas a autenticação no site, a escoragem em lote (carteira escorada) e a cobertura de clientes com poucas compras. Ficam como próximos passos:

- **Validação temporal:** treinar com um período e avaliar no seguinte, para medir a previsão de atrasos futuros
- **Model Serving:** servir o modelo registrado por um endpoint do Databricks, sem cópia do .pkl
- **Automação:** agendar 02 → 03 → 08 em um Databricks Job e gerar os números do painel a partir das tabelas
- **Explicabilidade:** mostrar no simulador quais variáveis mais pesaram em cada score
- **Painel de BI:** conectar a tabela carteira_escorada à ferramenta de BI da empresa

22. Responsáveis

Tabela 16 - Responsáveis

Nome	Função
Ana Elisa	Engenharia de Features / EDA
Arthur Nunes	Modelagem
Bruno Araujo	Pipeline, Databricks e Versionamento
Caio Corrá	Site / Deploy
Lucas Camelo	Monitoramento / Avaliação
Nicole Fava	Documentação / Testes

Fonte: Autoria própria

23. Aprovação

Espaço reservado para assinatura dos responsáveis pela validação e aprovação formal deste documento.

Nome	Cargo	Data	Assinatura
Arthur Nunes dos Santos	CEO		